

BigData Assignment 2 Report

Ilia Mistiurin

April 2025

1 Initial ideas and situation analysis

The coding part of this assignment copies the one from core IR course of the last semester, therefore I need to integrate my knowledge of simple search engine into the distributed nature of MapReduce, PySpark & HDFS, YARN.

I will state that my decisions and conclusions were made based on the results I have got or experienced. I run this assignment on MacOS system with 14Gb of RAM for the Docker engine, 100GB of hard drive space and 8 CPU cores: with this setup I used 1 Cassandra node, 2 Hadoop nodes: 1 master and 1 slave one. With these nodes I had resources left for approximately one more container, but it did the difference of degrading performance.

By running nodes in actually pseudo-distributed nodes the YARN resource manager was not able to allocate resources for PySpark applications: I made all configs to be able to run via YARN, but the .py files use local way of running it - could be solved by running nodes in real distributed system.

2 Methodology

2.1 Preparation

First of all, I have made the Dockerfile for packing env and package manager in it (it still uses the requirements as it was requested, but in different place and single time for the whole assignment). I see this as a better approach of managing dependencies, since we already introduced instability and bad practices by binding /app directory, therefore I made the python environment more robust this way. All paths to the env were managed in the .sh files.

Moreover, I slightly modified docker-compose with explicit platform usage and open-port for slave node for checking its tasks and logs from dashboard.

2.2 Indexing

The indexing was done as proposed by using the MapReduce. I divided the "load" of preprocessing input docs and inserting the results into Cassandra in 2 pipelines: first one parse-prepare data, second one with identity-mapper just

puts the output of the 1st reducer into the Cassandra. Idea was to utilizing the batch inserting, but I faced errors with the input size (5Kib as default) and the increase if this parameter the performance degrades for me.

The MapReduce Pipelines were made and remade several times: without 'understanding' what part of mapper should be reduced with or to what, the great number of such pipelines beeing created, with hard work putting their output to the one place, like Cassandra. I used the splitting with keeping the first word as a type indicator what action I should do with the input. This approach actually could use single MapReduce Pipeline, but for testing reasons I had splitted it into two parts (loosing quite a lot of performance)

I used 2 Cassandra tables with the following schema that will be used for bm25 calculation:

- table for term based information - term_vocab (term text, doc_id text, tf counter, PRIMARY KEY (term, doc_id)) This table holds quite a lot of data / stats that usually should be used at the same time
 - term is term or word from the document content
 - doc_id is the id of the document (for future compatability it is a string)
 - tf is a term frequency of the term if the documnet with the doc_id number
 - term with documnet id is a logical way to store documents: by utilizing Cassandra partions we easili find documents for terms in the user's query
- table - global_doc_stats (category text, key text, value float, doc_name text, PRIMARY KEY (category, key)) This table made for retrieving the document stats when we have it's id like after the query in previous table. These tables potentially could be united, but I see it as enormouse data duplication problem, considering the possible number of unique term (could easily tend to infinity).

I had to store average len of docs as vell as it's total number, but I didn't want to crate separate table just for 2 values, therefore I used sa partitioning key a predefined 'keys' as meta-doc. The documents as clustering key - no case will occure with the same document having more than one entry - this is the reason why it's quite uneffective to store this data in column databasse.

- category is a predefined category
- key is document id or a statistics identifier
- value the documnet len or the statistics
- doc_name is the original document name. Only needed for the top 10 output

Overall the code in the MapReduce Pipelines is not complicated to explain it - all ideas and logic behind it is above.

2.3 Ranking

The PySpark in my system in pseudo-distributed mode is not supported by YARN - YARN miscalculates resources for the tasks, therefore python code has overriding of yarn that is specified in the spark-submit arguments/

2.3.1 BM25 Ranking

The BM25 is just a quite good metric to use for short-to-medium queries for finding relevant documents. The calculation itself not that interesting to explain (just formula), considering that we had to implement it several times in the past courses.

The PySpark logic for retrieving data ideas (code in the repo, I do not think that I should rewrite PySpark docs here for it):

1. Check if terms - documents exists, if not - print this info to user
2. Parsing query by regex -> unique terms and making request to database with filter per these queries: terms are partition keys, that way we extract only the information that we actually need, using most Cassandra capabilities
3. Get document and global stats with filtering of the document's lengths that are in the 'terms' table we extracted in previous step
4. Apply scoring logic and calculating scores for documents that we have left
5. Get top-10 and print with extracted document names (during report writing I realized that the same trick as in step 2 could be done instead of my way of getting document names)

3 Demonstration

3.1 Running Repository

The docker compose used to run code in the repository. Changing query / data source is done manually.

For running print in the terminal in the project root: docker compose up -build

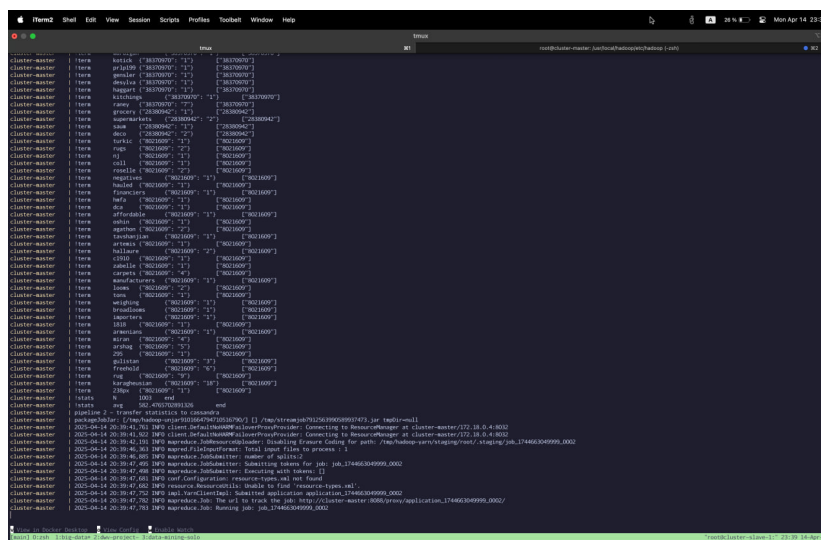
Adding documents done by adding a zipped parquet file in the /app folder, by default it is a.paquet.zip -> code will automatically unzip it and extract the data into the /app/data folder, that to the hdfs partition for future steps

In the app/app.sh in the end change the query that you need to find in docs.

3.2 Screenshots & Descriptions

I use fullscreen screenshots, but the full screenshot of 4K resolution screen not the best in copy-pasting into documents.

After MapReduce Pipeline 1 you can see 'cat' of the partition's content it provided to the Pipeline 2.



(See below) The second query found documents related to electronic music quite well for short query: all results are music bands or tracks. The next case is harder query "Electronic and hard rock music".

Overall the quality of the engine could be increased by better tokenization than word-parsing (stop words removal, etc) and these steps will have huge performance boost - but it is quite hard to use them in the MapReduce tasks.

